

# Lecture 13: Measuring performance

CMPE I20: Computer Organization & Architecture

Olivia Weng

# Logistics

- Reminder: check-in with professor is 1% of your final grade
  - Sign up [link](#)
- HW2 due this today (!), October 1st, 11:59pm
- HW2 Quiz next Tuesday

HW3 out today

# Measure performance

- I want my program to run faster. I want it to speed up!

# How to speed up a program?

- In the 1970s?

# How to speed up a program?

- In the 1970s?
  - Answer: Wait a year or two and buy a faster computer

# Program's speed determined by computer speed

- Computer speed often determined by its clock frequency

# Program's speed determined by computer speed

- Computer speed often determined by its clock frequency
- Clock frequency: number of CPU cycles per second
  - Measured in Hertz (1 / s)
  - Determines how much "work" a computer does per second

## Back in the day...

- A CPU's **clock frequency** increased by **~30-50% every year**
- Higher clock frequency → more work done per second → program runs in less time!

| Year | Clock Freq. (MHz) |
|------|-------------------|
| 1974 | 2                 |
| 1978 | 5                 |
| 1979 | 8                 |

# Example: Computing program latency — *time takes to execute*

- Example program: GPT-2 takes **3 billion cycles** to process 1 token
  - 3 billion =  $3 \times 10^9$

$$\frac{3 \times 10^9 \text{ cycles}}{2 \times 10^6 \frac{\text{cycles}}{\text{s}}} = 1.5 \times 10^3 \text{ s} \approx 25 \text{ min}$$

$$\frac{3 \times 10^9 \text{ cycles}}{5 \times 10^6 \frac{\text{cycles}}{\text{s}}} = 0.6 \times 10^3 \text{ s} = 10 \text{ min}$$

| Year | Clock Freq. (MHz) |
|------|-------------------|
| 1974 | 2                 |
| 1978 | 5                 |
| 1979 | 8                 |



## Example: Computing program latency

- Example program: GPT-2 takes 3 billion cycles to process 1 token
  - 3 billion =  $3 \times 10^9$

| Year | Clock Freq. (MHz) | Program latency                                        |
|------|-------------------|--------------------------------------------------------|
| 1974 | 2                 | $1.5 \times 10^3 \text{ s} \approx 25 \text{ min}$     |
| 1978 | 5                 | $0.6 \times 10^3 \text{ s} \approx 10 \text{ min}$     |
| 1979 | 8                 | $0.375 \times 10^3 \text{ s} \approx 6.25 \text{ min}$ |

Let's run an experiment!

# How did clock frequencies **increase** every couple years?

- Transistors got **smaller** year to year



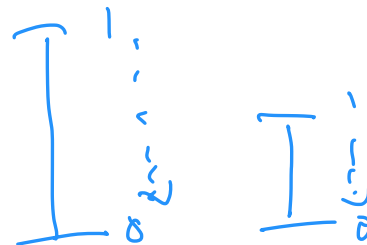
- Cram **twice** as many transistors on **same-size CPU** every **~2 years**
  - Known as **Moore's Law**

# How did clock frequencies **increase** every couple years?

- Transistors got **smaller** year to year
- Cram **twice** as many transistors on **same-size CPU** every **~2 years**
  - Known as **Moore's Law**
- **Key:** Twice as many transistors → but **same power consumption**
  - Known as **Dennard Scaling**
  - Prevents chips from **overheating**

# How did clock frequencies **increase** every couple years?

- A smaller transistor has a smaller **switching delay**



- **Switching delay** determines a CPU's clock frequency
  - Smaller delay → higher frequency!

# How to speed up a program?

Dennard scaling let us increase clock frequencies every  $\sim 2$  years at same power consumption

# How to speed up a program?

Dennard scaling let us increase clock frequencies every ~2 years at same power consumption

- In the 1970s?
  - Answer: Wait a year or two and buy a faster computer
- In the 1980s?

wait

+ buy the newest computer

- In the 1990s?

wait

- In the 2000s?

...

# Dennard Scaling ended around 2004-2005

- Voltage **stopped scaling down** with transistor size
  - Lowering it anymore caused transistors to **malfunction**

# Dennard Scaling ended around 2004-2005

- Voltage **stopped scaling down** with transistor size
  - Lowering it anymore caused transistors to **malfunction**
- Increasing **clock frequency** led to increased **power consumption & overheating**

# Dennard Scaling ended around 2004-2005

- Voltage **stopped scaling down** with transistor size
  - Lowering it anymore caused transistors to **malfunction**
- Increasing **clock frequency** led to increased **power consumption & overheating**

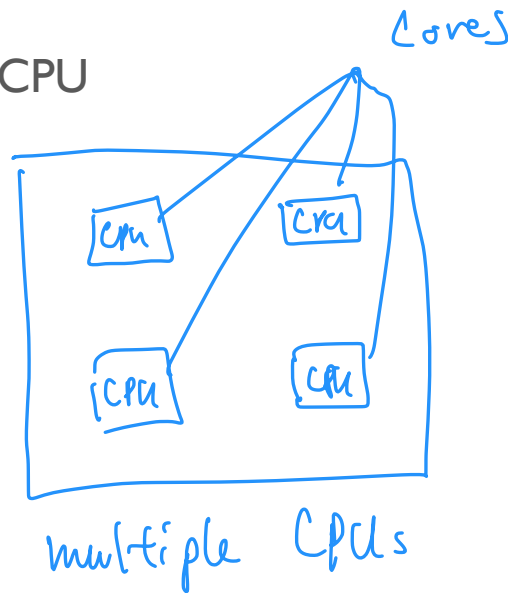
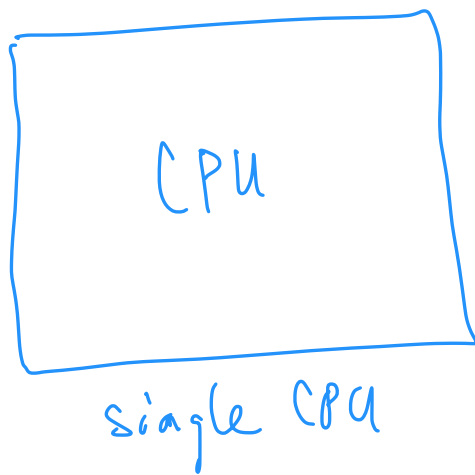
Chips could **no longer** increase frequency **without overheating**

With the end of Dennard Scaling, how to speed up programs?

- In the 2000s?
- In the 2020s?

# With the end of Dennard Scaling, how to speed up programs?

- Idea: Have multiple CPUs rather than a single CPU





Which programs benefit from multi-core computing?

→ have tasks that can be done in parallel

a video streaming app — diff users  
— streaming multiple videos

graphics rendering



Which programs do not benefit?

compiler

→ that need to run sequentially

↳ serially

bank transactions

# How much **speedup** can we get from multi-core?

- Not all parts of a program can be **parallelized**

# How much **speedup** can we get from multi-core?

- Not all parts of a program can be **parallelized**

- Many programs have **serial** tasks

↳ sequentially

# How much **speedup** can we get from multi-core?

Amdahl's law tells us the **speedup** we can get with **n** cores

$$T_{old} = 10s \Rightarrow \text{speedup} = \frac{10s}{5s} = 2x$$
$$T_{new} = 5s$$

- **speedup** = time\_old / time\_new
- given **p** percent of the program is parallelizable

$$\frac{T_o}{T_n} = \frac{T_s + T_p}{T_s + \frac{T_p}{n}}$$

$$T_s = T_o(1-p)$$
$$\underline{T_p} = T_o \cdot p$$

$$\Rightarrow \frac{T_o(1-p+p)}{T_o(1-p) + \frac{p}{n}}$$

$$\Rightarrow \boxed{\frac{1}{(1-p) + \frac{p}{n}}}$$

$$\Rightarrow \frac{T_o(1-p) + T_o \cdot p}{T_o(1-p) + \frac{T_o \cdot p}{n}}$$

# The principle of Amdahl's Law

- By **Gene Amdahl** (one of the designers of IBM 360)
- Optimizations **do not** (generally) uniformly affect the entire program
  - The more **widely applicable** a optimization is, the more **valuable** it is
  - **Limited applicability** can (drastically) **reduce** the impact of an optimization

This the most important thing you will learn in this course!

# Amdahl's Law Examples

$$\frac{1}{(1-p) + \frac{p}{n}}$$

Given a program that is 80% parallelizable, what is its speedup with

- 2 cores?

$$\frac{1}{(1-0.8) + \frac{0.8}{2}} = \frac{1}{0.2 + 0.4} = \frac{1}{0.6} = \frac{10}{6} = \boxed{1.67 \times}$$

- 4 cores?

$$\frac{1}{(1-0.8) + \frac{0.8}{4}} = \frac{1}{0.2 + 0.2} = \frac{1}{0.4} = \frac{10}{4} = \boxed{2.5 \times}$$

- 32 cores?

# Amdahl's Law Examples

Given a program that is 80% parallelizable, what is its speedup with

- 2 cores? 1.67 X

- 4 cores? 2.5 X

- 32 cores? 4.4 X

Amdahl's Law gives us the **maximum speedup** of a program

- Using calculus...

$$\lim_{n \rightarrow \infty} \frac{1}{(1-p) + \frac{p}{n}} = \boxed{\frac{1}{1-p}}$$

# Amdahl's Law Examples

$$\frac{1}{1-p}$$

What is the **maximum speedup** of a program that is

- 80% parallelizable?

$$\frac{1}{1-0.8} = \frac{1}{0.2} = \boxed{5\times}$$

- 50% parallelizable?

- 99% parallelizable?

# Amdahl's Law Examples

What is the **maximum speedup** of a program that is

- 80% parallelizable? **5 X**
- 50% parallelizable? **2 X**
- 99% parallelizable? **100 X**

# GPT-2 Present Day

- GPT-2 is **fast** now because it has big parallelizable portions
- CPU → GPU (ultra parallel processor)
- Why we can have a conversation with GPT in real-time

Is **faster** always better?

# Match (best) performance metric to domain

- Performance Metrics

- Network Bandwidth (MB/sec)
- Network Latency (ms)
- Frame Rate (frames/sec)
- Throughput (ops/sec)

|   | Fortnite<br>(online) | Call of Duty<br>(single player) | Watching<br>Netflix | Google<br>Server<br>Farm |
|---|----------------------|---------------------------------|---------------------|--------------------------|
| A | 4                    | 3                               | 1                   | 2                        |
| B | 4                    | 1                               | 3                   | 2                        |
| C | 2                    | 1                               | 3                   | 4                        |
| D | 2                    | 3                               | 1                   | 4                        |
| E | None of the above    |                                 |                     |                          |

# References

- Hennessy, John L., and David A. Patterson. *Computer architecture: a quantitative approach*. Elsevier, 2025.
- CPU DB. [http://cpudb.stanford.edu/visualize/clock\\_frequency.html](http://cpudb.stanford.edu/visualize/clock_frequency.html)
- Inference characteristics of Llama. <https://cursor.com/blog/llama-inference>
- Future chips will be hotter than ever.  
[https://spectrum.ieee.org/hot-chips#:~:text=Summary%20\\*%20Engineers%20continued%20to%20increase%20transistor,could%20also%20help%20in%20heat%20management%2C%20too.](https://spectrum.ieee.org/hot-chips#:~:text=Summary%20*%20Engineers%20continued%20to%20increase%20transistor,could%20also%20help%20in%20heat%20management%2C%20too.)